



ElectroDSL

AI-Powered Electrical Schematic Editor

Beta Release v2.0

Open Source · MIT License · Runs in any browser · No installation required

"Write the circuit. See the schematic." — ElectroDSL treats electrical drawings the same way LaTeX treats documents: plain text is the source of truth, the visual rendering is just one view of it.

Type	Browser-based web application (HTML + CSS + Vanilla JS)
License	MIT — free to use, modify, distribute, and sell
Status	Beta v2.0 — functional, open for community contributions
Dependencies	None — zero npm packages, zero build step
Browser support	Chrome 90+, Firefox 88+, Edge 90+, Safari 15+
AI Integration	Anthropic Claude API (claude-sonnet-4-6) — optional
Symbol count	21 built-in + 41 building electrical (importable)
Export formats	SVG, PDF, SPICE netlist (.cir)
File format	Plain text .edsl — human-readable, Git-friendly

1. What is ElectroDSL?

ElectroDSL is a **domain-specific language (DSL)** and **interactive editor** for creating electrical schematic diagrams. It combines the precision of code-based drawing with the convenience of an interactive graphical canvas — letting you work in whichever mode suits you.

Unlike traditional EDA tools (KiCad, Eagle, Altium) which are heavy desktop applications focused on PCB design, ElectroDSL is designed for **schematic communication** — quickly expressing how a circuit works, documenting building electrical installations, teaching electronics, or prototyping ideas.

The core concept is that a circuit description is **just text**. This means your schematics can be version-controlled with Git, diffed line by line, generated by AI, included in documentation, and shared as a simple text snippet.

```
// This is a complete, valid ElectroDSL schematic:
CIRCUIT "Voltage Divider" {
  V1:  VSource(12V,DC) at (0, 5) label="12V Supply"
  R1:  Resistor(10k)   at (3, 6) label="R1"  value="10kΩ"
  R2:  Resistor(10k)   at (3, 4) label="R2"  value="10kΩ"
  VOUT: Node           at (5, 5) label="Vout=6V"
  GND1: Ground         at (0, 3)
  GND2: Ground         at (6, 3)

  wire V1.p -> R1.1
  wire R1.2 -> R2.1
  wire R1.2 -> VOUT.1
  wire R2.2 -> GND2.1
  wire V1.n -> GND1.1
}
```

This text produces a complete rendered schematic — exportable to SVG, PDF, and SPICE.

1.1 Who is it for?

Electrical engineers	Quickly document circuits, create installation diagrams, share schematics as text snippets in emails or chat
Electrical contractors	Draw building electrical installations (consumer units, ring mains, lighting circuits, fire alarms) using the building s
Electronics hobbyists	Prototype and document Arduino, Raspberry Pi, and embedded circuits without installing heavy EDA software
Students & educators	Learn circuit theory by writing circuits as code; AI assistance helps explain what each component does
Technical writers	Embed schematic code in documentation; schematics are text so they render in any markdown or doc pipeline
AI developers	Use the structured DSL format as a reliable output format for LLM circuit generation tasks

2. Feature Overview

2.1 Core Capabilities

DSL Code Editor	Syntax-highlighted editor with keyword colouring (blue=keywords, green=types, yellow=values, purple=pins). Wires are rendered in the background.
Interactive Canvas	Drag components to reposition (DSL updates automatically). Draw wires by clicking pin-to-pin with optional bend points.
AI Generation	Describe a circuit in plain English — the AI generates the complete DSL. Works with the built-in button (Anthropic) or your own API key.
Live Sync	Canvas and editor stay in sync in both directions: editing code re-renders the canvas; moving a component on the canvas updates the code.
Net Awareness	Every wire segment is automatically assigned a net name (GND, VCC, N001...). Net list panel shows all connections and their net names.
Auto Layout	Components without coordinates are automatically placed using a BFS graph layout algorithm based on wire connections.
Smart Grid	Adjustable cell size (×0.5 to ×2) and subdivision (1/2/4/8 per cell). Snap cursor crosshair shows exact snap point.
Wire Routing	Wires route orthogonally with smooth Bézier corners. Click empty space during drawing to add bend points. Click a pin to finish.

2.2 Canvas Tools

Tool	Key	Description
Select & Move	V	Click to select. Drag to move (snaps to grid). Double-click to rotate 90°. Delete key removes selected component and all wires connected to its pins.
Wire Draw	W	Click a pin (green ring appears) to start. Click empty space to add bend points — the route is remembered. Click target pin to finish.
Join Wire	J	Click on any existing wire to insert a junction Node at that point, splitting the wire into two segments connected through the junction.
Rotate	R	Click any component to rotate 90° clockwise. Pin positions and wire connections rotate with it.
Delete Wire	E	Click any wire segment to delete just that wire, leaving both connected components in place.
Delete Component	D	Click any component to delete it along with all wire connections to its pins.

3. Symbol Libraries

ElectroDSL ships with two symbol libraries. The built-in library is always available. The building electrical library is imported once from the JSON file and stored in browser localStorage.

3.1 Built-in Electronic Symbols (21 symbols)

Sources (2)	VSource — DC/AC voltage source · ISource — current source
Passives (3)	Resistor · Capacitor · Inductor
Diodes (4)	Diode · LED · Zener · Schottky
Transistors (6)	NPN BJT · PNP BJT · NMOS · PMOS · JFET · Thyristor/SCR
Amplifiers (1)	OpAmp — 5-pin with V+/V- supply rails
Connections (3)	Ground · VCC power rail · Node / net junction
Misc (2)	Switch · Transformer

3.2 Building Electrical Symbols (41 symbols, importable JSON)

Socket Outlets (4)	Single 13A · Double 13A · Fused Connection Unit · Weatherproof IP44
Switches (6)	1-Way · 2-Way · Intermediate · Dimmer · PIR Motion · Timer
Lighting (4)	Lighting Point · Emergency Luminaire · Exit Sign · Fixed Downlight
Distribution (7)	Consumer Unit · MCB · RCBO · RCD · Isolator Switch · Sub-DB · Smart Meter
Fire & Safety (5)	Smoke Detector · Heat Detector · Fire Bell/Sounder · Break Glass MCP · CO Detector
Data & AV (3)	TV Aerial Socket · RJ45 Data Socket · BT Telephone Socket
Security (2)	Access Control Reader · CCTV Camera
Wiring Accessories (5)	Door Bell · Cooker Control · Shower Pull Cord · Extractor Fan · SWA Termination
Heating & Controls (2)	Room Thermostat · Boiler Controller
Specialist (3)	EV Charging Point (7kW) · Solar PV Inverter · Battery Storage (BESS)

3.3 Custom Symbol Import

Any symbol can be imported as a JSON file via Library → Import JSON. Custom symbols are stored in browser localStorage and persist between sessions. They can be exported and shared with others. The included AI Prompt Guidelines PDF explains how to prompt any LLM to generate new symbol JSON files for specialist equipment.

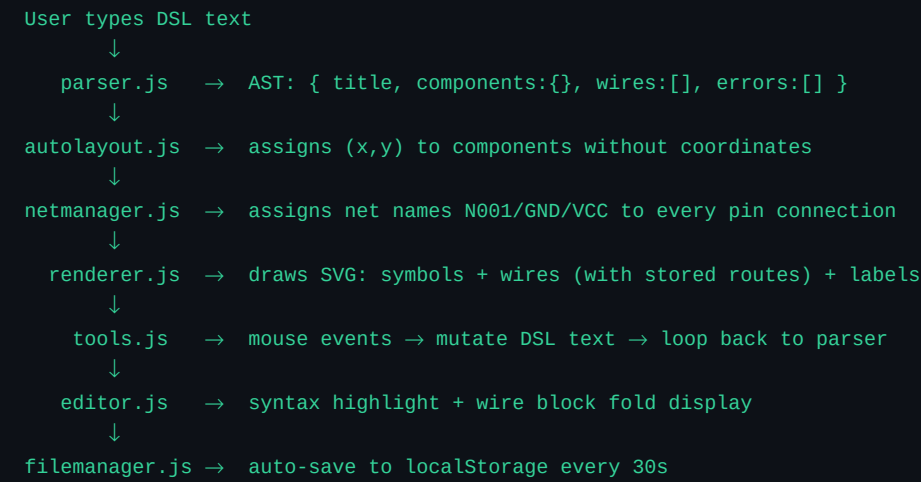
4. Technical Architecture

ElectroDSL has zero external dependencies. Every capability is implemented in plain HTML, CSS, and vanilla JavaScript. There is no build step, no npm, no bundler, no server. Just files.

4.1 Module Structure

Module	Lines	Responsibility
gridmanager.js	~120	Grid cell size, subdivision, snap calculation, SVG pattern update, snap cursor
symbols.js	~420	SVG symbol drawings for all 21 built-in types, pin coordinate definitions
library.js	~220	Custom symbol storage in localStorage, import/export JSON, DSL snippet generator
parser.js	~140	DSL tokeniser, AST builder, pin alias resolver, route attribute parser
netmanager.js	~110	Union-Find net grouping, net name assignment, net table HTML renderer
renderer.js	~750	SVG canvas renderer, pin-based wire drawing, component drawing, self-contained SVG exp
tools.js	~680	Mouse interaction engine: drag, wire draw, rotate, delete, join, snap cursor
autolayout.js	~90	BFS-based automatic component positioning from wire connectivity graph
spice.js	~120	SPICE netlist generator, KiCad netlist generator
ai.js	~80	Anthropic Claude API integration, system prompt with full symbol reference
editor.js	~320	Syntax-highlighted editor, wire block folding, gutter, preview panel
filemanager.js	~200	Save/open .edsl files, recent projects, auto-save, dirty-state tracking
app.js	~1000	Application controller, wires all modules together, event handlers

4.2 Data Flow



5. Getting Started

5.1 Instant Start (Recommended)

- 1. Download electrodsl.zip and extract to any folder
- 2. Double-click index.html — opens in your default browser
- 3. Load an example from the Examples tab in the left panel
- 4. Start drawing — drag components, draw wires, or type DSL code

5.2 Using AI Generation

Type a circuit description in the AI bar at the top and click Generate. The AI writes the DSL and renders it immediately. For best results, be specific:

```
Good: "3-phase ring main with 6 double sockets, RCBO B32A 30mA protection,
      kitchen FCU spur, all earths to main GND"

Good: "NPN amplifier, 12V supply, 10kΩ/2.2kΩ bias resistors, 4.7kΩ collector,
      1kΩ emitter, 10μF coupling capacitors, 100μF bypass"

Vague: "make a circuit" ← AI will guess, results vary
```

5.3 Building Electrical Drawings

For UK building electrical installations, import the building symbol library first:

- 1. Click Library tab (left panel)
- 2. Click Import JSON
- 3. Paste or drag building_electrical_symbols.json
- 4. Symbols appear in palette immediately
- 5. Use AI: "1-way lighting circuit with MCB B6A, switch, LED lamp, emergency light"

5.4 Deployment Options

Local file	Open index.html directly — works fully offline except AI generation
GitHub Pages	Push folder to GitHub repo, enable Pages in Settings → Pages
Netlify	Drag the folder to netlify.com/drop — live in 30 seconds
Vercel	Run: npx vercel --prod in the folder
Any web server	Copy files to any web server serving static files — Apache, Nginx, etc.
Docker	Serve with: docker run -p 80:80 -v \$(pwd):/usr/share/nginx/html nginx:alpine

6. Open Source & MIT Licence

ElectroDSL is released under the MIT Licence — one of the most permissive open-source licences available. This means:

✓ Use freely	Use ElectroDSL for any purpose — personal, commercial, educational, government
✓ Modify freely	Change any part of the code — the editor, symbols, DSL syntax, UI, everything
✓ Distribute freely	Share copies with anyone, include in products, distribute on any platform
✓ Sublicence	Incorporate into proprietary software or other open-source projects
✓ No royalties	No fees, no licencing costs, no usage limits now or in the future
✓ No warranty required	No obligation to provide support, updates, or bug fixes
! Attribution	Keep the copyright notice in the source files — that is the only requirement

```
MIT License

Copyright (c) 2025 ElectroDSL Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
```

7. Contributing & Community

ElectroDSL is an open project welcoming contributions of all kinds — from fixing a typo in the docs to adding a major new feature. The codebase is deliberately simple (plain JS, no frameworks) so anyone comfortable with web development can contribute immediately.

7.1 How to Contribute

1. Fork the repository on GitHub
2. Create a branch: `git checkout -b feature/my-improvement`
3. Edit files in `src/` — open `index.html` to test instantly, no build needed
4. Write a clear commit message describing what changed and why
5. Submit a Pull Request with a description and screenshots if visual

7.2 Suggested Improvements (Roadmap)

Area	Priority	Description
Undo/Redo	High	Full history stack with Ctrl+Z / Ctrl+Y. Every DSL mutation should push to a history array.
Wire routing	High	Smarter orthogonal routing that detects and avoids component bodies. A* pathfinding on the grid.
Multi-select	High	Rubber-band selection rectangle, move/delete multiple components at once.
Component snap	Medium	Snap component to grid during drag (not just on release) for smoother interaction.
Properties panel	Medium	Sidebar showing selected component params, editable fields that update DSL directly.
Wire labels	Medium	Click a wire to name/edit its net label directly on the canvas.
More symbols	Medium	Automotive, RF, relay ladder logic, IEC 60617, IEEE/ANSI standard symbol sets.
Hierarchical	Medium	Sub-circuit blocks — define a circuit once, reuse it as a symbol elsewhere.
BOM export	Medium	Bill of Materials as CSV/Excel — component ID, type, value, quantity.
Monaco Editor	Low	Replace textarea with VS Code's Monaco editor for full IntelliSense and error squiggles.
PWA/Offline AI	Low	Service worker + local LLM (WebLLM) for fully offline AI generation.
Collaboration	Low	WebSocket-based multi-user editing — like Figma for schematics.
Simulation	Low	Link generated SPICE netlist to online simulators (PartSim, Falstad, etc.).
PCB link	Low	Attach KiCad/EasyEDA footprints to symbols for schematic-to-PCB workflow.
Plugin API	Low	Public JS API for third-party tools to extend the parser, renderer, and tools.

7.3 Good First Issues for New Contributors

Add more examples	Write 5–10 new CIRCUIT examples (.edsl files) for the Examples tab — lighting, motor control, audio, sensors
Symbol validation	Compare built-in symbols against IEC 60617 / IEEE Std 315 and document any differences
New building symbols	Add symbols for: underfloor heating, HVAC, solar thermal, industrial 3-phase sockets
Error messages	Improve parser error messages to be more helpful ("Did you mean .1 instead of .left?")
Keyboard shortcuts	Add missing shortcuts: Ctrl+Z undo, Ctrl+A select all, Ctrl+D duplicate component
Mobile/touch	Improve touch events so the wire tool works on tablets with a stylus
Accessibility	Add ARIA labels, keyboard navigation, and screen reader support to the canvas
Tests	Write automated tests for the parser, net manager, and symbol library using browser test runners

8. Known Beta Limitations

The following limitations are known in v2.0 and targeted for community improvement:

No undo/redo	Ctrl+Z does not work. Workaround: use the file auto-save and reload from Recent Projects.
Wire routing	Wires route through component bodies. The orthogonal L-router does not avoid symbols.
No multi-select	Components must be moved one at a time. Group operations not yet supported.
Pin snap offset	In some zoom levels, the pin snap target may be slightly offset from the visual pin tip. Workaround: zoom in to 150%.
Large schematics	Performance degrades noticeably above ~100 components. Hierarchical sheet support would solve this.
Mobile touch	Touch/stylus wire drawing works but is not optimised. Desktop use is recommended for v2.0.
AI requires internet	The built-in AI button needs internet access to reach the Anthropic API. Offline users should use the copy-paste prompt.
SPICE accuracy	The SPICE export generates a structural netlist but node numbering is simplified. For accurate simulation, the netlist must be manually edited.

ElectroDSL Beta v2.0

"Write the circuit. See the schematic."

Open Source · MIT License · No installation required · Runs in any browser